VoiceOps 2026: ZentroMart Voice Support Agent

Capstone report | Mrittika Kunti, Institute of Engineering and Management, Kolkata | Code: github.com/Mrittika666/voiceops-zentromart

1. What I built

A customer support agent for the fictional store ZentroMart that answers from company documents, holds a multi-turn conversation, uses the order and ticket tools, and can take a call. One customer turn flows through: **Gnani STT** -> **agent** (LLM + RAG over the knowledge base + lookup_order / create_ticket + conversation state) -> **Gnani TTS**. The brain (Steps 1 to 3) is text only; voice (Step 4) wraps it without changing it. Code layout: `src/kb.py` (chunking, embeddings, retrieval), `src/agent.py` (state, tools, prompting), `src/voice.py` (STT/TTS), `run.py` (batch run over a folder of audio).

2. Step 1: plain LLM baseline

A system prompt makes the model ZentroMart's support assistant, with the first reply telling the customer it is an AI assistant. It saw no documents. I ran all 52 practice questions and scored each answer automatically against the must_include and must_not_include columns (keyword match; see Section 6 for the limits of this scorer). Model: Groq `openai/gpt-oss-120b`, used for both Step 1 and Step 2 so the comparison is fair.

3. Step 2: grounding with RAG

- **Chunking:** the 21 Markdown documents were split at section headings into 93 chunks. Each chunk keeps its [SECTION-ID], document status and effective_from date, so every answer can be traced to its sources.
- **Index:** multilingual MiniLM sentence embeddings (handles Hindi and Hinglish queries) in a FAISS inner-product index.
- **Precedence:** chunks marked superseded (the old returns policy v3) are dropped, so the current policy v4 wins.
- **Answering:** top 5 chunks go into the prompt; the model must answer only from them and say it does not have the information (and offer a human) when the documents do not cover the question. The section IDs used are saved for every answer.

Practice questions (52)	Step 1: no documents	Step 2: RAG
Total passed	17 / 52	**37 / 52**
Answerable questions (47)	17 / 47	32 / 47
Not covered by documents (5): correctly says it does not know	0 / 5	**5 / 5**

What I learned: without documents the model answers confidently and never says it does not know (0 of 5). Grounding more than doubled the pass count and fixed the unanswerable cases completely. I have not yet analysed the remaining 15 failures question by question; that is the first thing I would do with more time.

4. Step 3: holding a conversation

The agent keeps explicit state per conversation: order ID, whether the caller is verified, failed verification attempts, chat history and the ticket ID. Rules enforced in code, not left to the model:

- The order ID is extracted from the message (digits may be spoken in groups, for example 8978 2837). `lookup_order` is called only with a phone suffix or pincode from the customer; order details reach the model only after verification passes, with at most one retry before a human takes over.
- When the model sets escalate=true, the code calls `create_ticket` and the reply says a ticket was created **only if created=True came back**. The model is told never to claim a ticket itself.
- Retrieval for each turn uses the last three customer messages, so follow-ups such as how long will that take work without repeating the order. The escalation guide is always included in the context, so value limits and handover rules are visible every turn.
- Today (20 October 2026), the business-day rule (Mon to Sat, no holidays from holidays.csv) and the calendar-day return windows are in the system prompt.

Practice conversations (23 conversations, 96 turns)	Result
First full run: text checks passed	73 / 96, zero API errors
First full run: ticket created exactly when expected	87 / 96

Practice conversations (23 conversations, 96 turns)	Result
After fixes (later rerun, only the 73 turns that completed before the API quota ran out)	59 / 73 text, 65 / 73 ticket

The second row set is not directly comparable to the first because the rerun was cut short by the provider's daily token limit; I report it as evidence the fixes did not break earlier behaviour, not as a clean improvement. The first six failing turns I inspected fell into four groups: the agent approved a claim above the 10,000 rupee limit, did not state exact dates, said it did not know a refund timeline that was in the documents (Hinglish queries missed the refund chunk), and did not escalate a repeated-contact complaint. I fixed them with prompt rules, always-on escalation chunks and keyword expansion of Hinglish queries.

5. Step 4: voice

- Gnani STT (en-IN, accepts mixed English and Hindi) in, Gnani TTS out (timbre-v2.5, voice Kaveri). Empty or unclear audio gets the reply: Sorry, I didn't catch that. Could you say it again?
- STT writes spoken numbers as words. A small parser converts them back to digits (for example four thousand five hundred thirty three becomes 4533), which made phone-suffix verification work by voice.
- Replies are written for the ear: short, plain text, years in dates, order numbers split into groups of four digits, and the long ticket ID is left out of the audio (it stays in the text output).
- `python run.py <audio_folder> --out <dir> --data <dataset>` processes every file, keeps files like PC002_1.wav, PC002_2.wav in one conversation, and saves the reply audio, the reply text, the retrieved section IDs and the tool calls (results.csv and results.json).
- **End-to-end test:** four synthetic customer audio clips (practice conversation PC002, generated with Gnani TTS, since no audio ships with the dataset). The agent asked for the order ID, then verification, verified the caller from the spoken digits, and escalated the damaged-TV replacement above the value limit with a real ticket. I have not yet tested with a real microphone recording of my own voice or with Hindi audio.

6. What went wrong and what I would fix

Problem	What happened	What I would do next
Free API quotas	Gemini free tier allows 20 requests per day, so my first Step 2 run produced 52 errors (and a misleading score of 11/52). I switched to Groq, which later hit its daily token limit during the conversation rerun.	Use a paid or higher-quota key for evaluation; cache retrieved chunks; add exponential backoff (done) and a model fallback (done).
Weaker fallback model	gpt-oss-20b approved a claim it should have escalated; qwen 27b handled the same test correctly.	Run the full 96-turn evaluation against each fallback model before trusting it.
Approximate scoring	My scorer is keyword matching on must_include and must_not_include. It can pass a wrong answer that contains the keyword and fail a correct paraphrase.	Add an LLM-as-judge check against reference answers and check retrieval against gold_evidence IDs.
Limited voice testing	Only synthetic English audio, one conversation.	Record my own voice, test Hindi and Bengali input with the matching language code, and add a retry when the transcript looks like noise.
Retrieval	Plain top-5 dense retrieval; Hinglish needs keyword expansion.	Add a reranker and hybrid keyword plus dense search.

7. Extra work beyond the brief

- Automatic model fallback when the main model hits its rate limit, so a batch run keeps going.
- Spoken-number parsing and ear-friendly reply formatting for voice.
- Retrieval keeps source section IDs for every reply, so each answer can be checked against the documents.